Práctica 06 — Convertir al personaje en un cuerpo físico

Programación de Videojuegos · SMR · FP Lumara Duración: 1 sesión · **Práctica previa:** 05

> ***Lo que aprendes hoy: la diferencia entre un dibujo que se mueve y un cuerpo que ocupa sitio.***

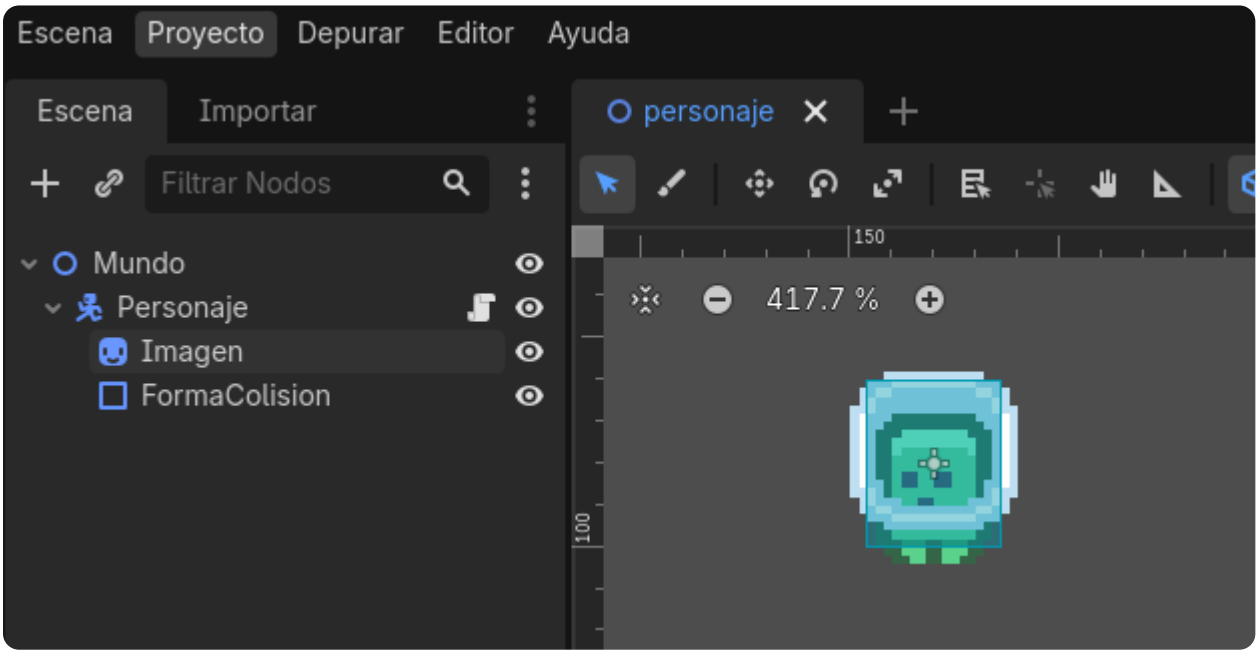
---

1. Qué vas a conseguir

Aviso por delante: **al ejecutar, esto se va a ver exactamente igual que la práctica 05**. El personaje se mueve con el teclado y ya está.

Lo que cambia está por dentro. Hasta ahora el personaje era un dibujo al que le cambiábamos la posición, y podía atravesar cualquier cosa porque no ocupaba espacio. A partir de hoy es un **cuerpo**: tiene una forma, ocupa sitio y puede chocar.

Sin esto, las rocas de la práctica 09, las plataformas de la 18 y los enemigos de la 20 son imposibles. Hoy se pone el cimiento.



"El rectángulo azul es lo que de verdad ocupa sitio; el dibujo solo se ve."

---

2. Materiales

Descomprime `practica_06_inicio.zip`, que trae el resultado de la práctica 05. Si tienes el tuyo terminado, también puedes seguir con él.

---

3. Los tres tipos de cuerpo de Godot

Nodo	Qué hace	Ejemplo
`StaticBody2D`	No se mueve nunca, pero frena a los demás	Una pared, una roca
`RigidBody2D`	Se mueve solo, obedeciendo a la física	Una caja que cae y rebota
`CharacterBody2D`	Se mueve cuando **tú** se lo dices, y respeta los obstáculos	El personaje del jugador

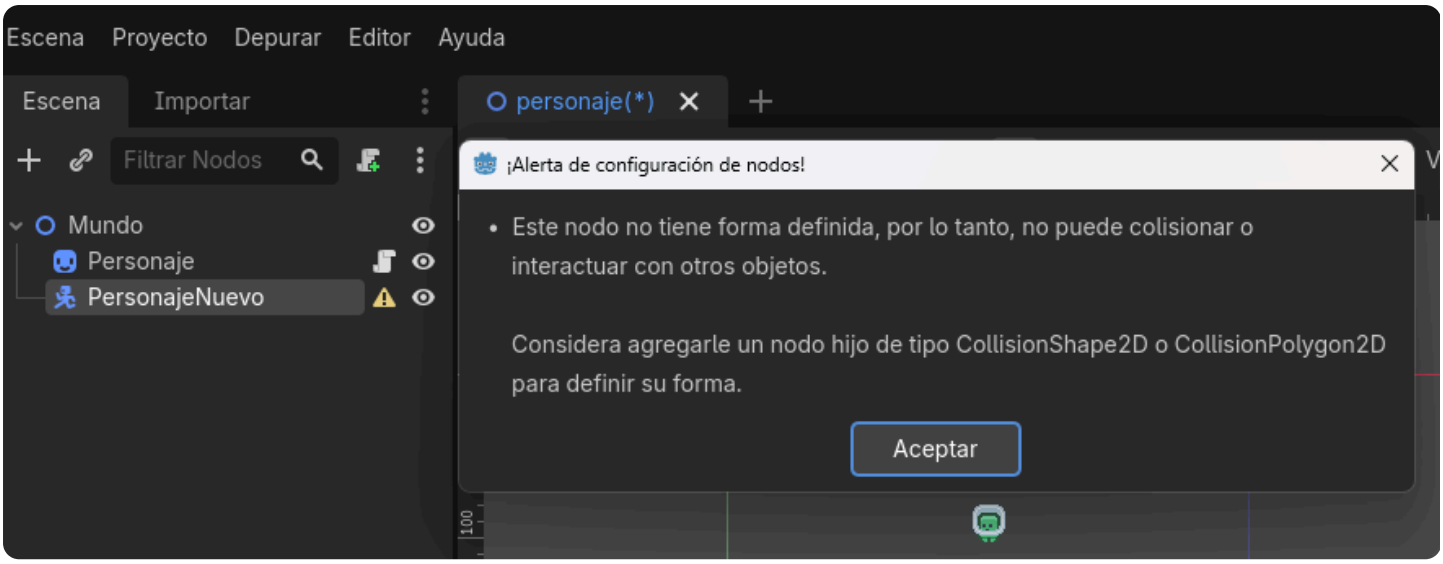
Nosotros usamos siempre `CharacterBody2D` para los personajes. El motivo es de control: en un videojuego quieres que el protagonista se pare en seco cuando sueltas la tecla, no que patine y rebote como haría un objeto físico de verdad.

---

4. Paso a paso

4.1 Crear el cuerpo

1. Selecciona el nodo `Mundo` y añade un hijo de tipo `CharacterBody2D`.
2. Llámalo `Personaje`. Como ya hay otro nodo con ese nombre, ponle de momento `PersonajeNuevo` y lo arreglamos en un momento.
3. Fíjate en que aparece un **triángulo amarillo de aviso** junto al nodo. Pincha encima y léelo: dice que este nodo necesita una forma de colisión.

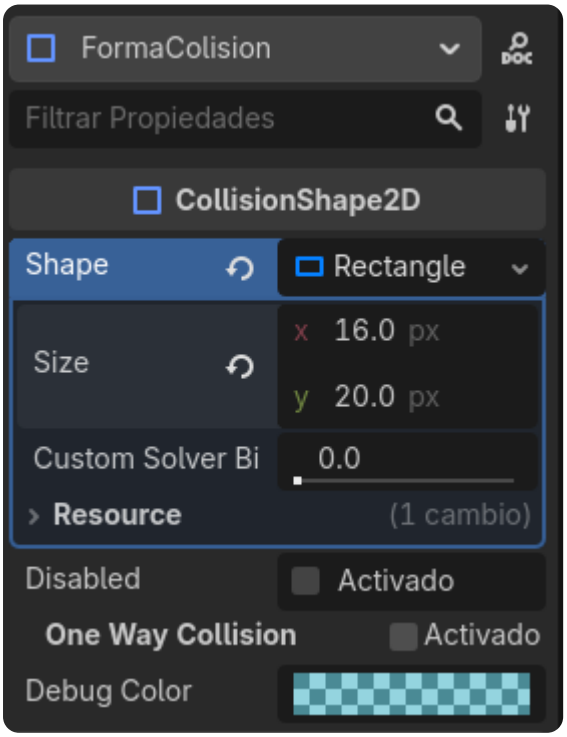


"Godot avisa: un cuerpo sin forma de colisión no sirve de nada."

4.2 Darle una forma

4. Con el cuerpo seleccionado, añade un hijo de tipo `CollisionShape2D` y llámalo `FormaColision`.
5. En el **Inspector**, la propiedad **Forma** pone `<vacío>`. Pincha y elige **Nuevo RectangleShape2D**.
6. Ajusta el tamaño a **16 x 20** aproximadamente. Puedes escribirlo en el Inspector o arrastrar los puntos rojos en el visor.

> ***La forma de colisión no tiene que coincidir exactamente con el dibujo. Suele hacerse un poco más pequeña que el sprite: así el jugador tiene la sensación de que el juego es generoso y no se queda enganchado en las esquinas.***



"Un rectángulo es suficiente para casi cualquier personaje 2D."

7. Comprueba que el aviso amarillo ha desaparecido.

4.3 Meter el dibujo dentro del cuerpo

8. En el panel **Escena**, arrastra el nodo `Personaje` (el `Sprite2D` de la práctica anterior) y suéltalo **encima de** `PersonajeNuevo`, para que quede como hijo suyo.
9. Renómbralo a `Imagen`.
10. Con `Imagen` seleccionado, pon su **Posición** a `0, 0`. La posición de un hijo se cuenta **desde su padre**, así que un cero significa "justo encima del cuerpo".
11. Ahora renombra `PersonajeNuevo` a `Personaje` y ponle **Posición** `160, 90`.

El árbol te tiene que quedar así:

```
Mundo
└── Personaje        (CharacterBody2D)   ← aquí va el script
    ├── Imagen        (Sprite2D)
    └── FormaColision (CollisionShape2D)
```

> ***Este patrón se repite el resto del curso. El cuerpo manda y se mueve; el dibujo y la forma van dentro y le acompañan. Cuando el cuerpo se mueve, sus hijos se mueven con él, gratis.***

4.4 Mover el script

12. El script sigue colgado de `Imagen`, que ahora ya no es quien manda. Selecciona `Imagen`, botón derecho, **Desconectar Script**.
13. Selecciona `Personaje` y añade el script `res://scripts/personaje.gd` (el que ya existe, no crees uno nuevo).
14. Cambia su contenido por:

```gdscript
extends CharacterBody2D

var velocidad := 100.0


func _physics_process(_delta: float) -> void:
    var direccion := Input.get_vector(
        "mover_izquierda", "mover_derecha", "mover_arriba", "mover_abajo")
    velocity = direccion * velocidad
    move_and_slide()
```

15. Guarda todo y ejecuta con **F6**.

---

5. El código, línea a línea

`extends CharacterBody2D` Ha cambiado, porque el script ya no vive en un `Sprite2D`. Si te olvidas de esto, Godot se quejará de que `velocity` no existe.

`func _physics_process(_delta: float)` Es la hermana de `_process`, pero se llama en cada paso de la **física**, no en cada dibujado. La física va a un ritmo fijo (60 veces por segundo) aunque los fotogramas suban o bajen.

Regla para todo el curso: si el código mueve un cuerpo, va en `_physics_process`. Para lo demás, `_process`.

`velocity = direccion * velocidad` `velocity` es una propiedad que **ya trae** el `CharacterBody2D`. No la hemos creado nosotros. Guarda a qué velocidad y en qué dirección se quiere mover el cuerpo, en píxeles por segundo.

Fíjate en que aquí no tocamos `position` en ningún momento.

`move_and_slide()` Es la función que mueve el cuerpo de verdad. Coge `velocity`, la aplica, comprueba si hay algo en medio y, si lo hay, **desliza** el personaje a lo largo del obstáculo en vez de dejarlo clavado.

Hace dos cosas más que conviene saber:

- **Multiplica por delta ella sola.** Por eso hoy el parámetro vuelve a llevar guion bajo: `_delta` ya no se usa.
- Guarda información sobre con qué ha chocado, que usaremos en prácticas posteriores.

---

6. Pruébalo

- [ ] El personaje se mueve igual que en la práctica 05.
- [ ] No hay ningún aviso amarillo en el panel Escena.
- [ ] En el editor se ve el rectángulo azul de la colisión alrededor del personaje.
- [ ] En el panel **Salida** no aparece nada en rojo.

---

7. La teoría en cinco líneas

- Un **cuerpo** es un nodo que ocupa espacio y puede chocar; un sprite solo se dibuja.
- Un `CharacterBody2D` necesita **sí o sí** un `CollisionShape2D` con una forma.
- La estructura habitual es: cuerpo como padre, imagen y forma como hijos.
- `velocity` guarda la velocidad deseada y `move_and_slide()` la aplica respetando obstáculos.
- El código que mueve cuerpos va en `_physics_process`, que corre a ritmo fijo.

---

8. Si no te funciona

Lo que ves	Qué pasa	Cómo se arregla
Triángulo amarillo en el nodo	Falta el `CollisionShape2D` o su forma	Añádelo y créale una forma
`Invalid assignment to property 'velocity'`	El script sigue con `extends Sprite2D`	Cámbialo a `CharacterBody2D`
El dibujo se queda atrás	El `Sprite2D` no es hijo del cuerpo	Arrástralo dentro en el panel Escena
El dibujo está descolocado	El hijo no tiene posición `0, 0`	Pon la posición del hijo a cero
Se mueve a velocidad rarísima	Han dejado `* delta` en `velocity`	`move_and_slide()` ya lo aplica
No se mueve nada	El script está en `Imagen` o en `Mundo`	Tiene que estar en el `CharacterBody2D`
Hay dos personajes en pantalla	Crearon un cuerpo nuevo sin mover el sprite antiguo	Debe quedar un solo `Sprite2D`, dentro del cuerpo

> ***Un aviso sobre el botón Cambiar Tipo. Godot permite cambiar el tipo de un nodo con el botón derecho. Es tentador para convertir el `Sprite2D` en `CharacterBody2D` de golpe, pero se pierden las propiedades del nodo antiguo, incluida la textura. Por eso en esta práctica creamos un nodo nuevo y movemos el sprite dentro.***

---

9. Retos

Verde · Cambia la forma de colisión por un `CircleShape2D` y ajusta su radio. ¿En qué se diferencia al moverse pegado a una esquina? De momento no se nota nada, pero apunta lo que crees que pasará.

Amarillo · Añade un segundo `Sprite2D` como hijo del cuerpo, con la otra imagen del personaje, y colócalo un poco desplazado ajustando su posición. Comprueba que al mover el cuerpo se mueven los dos dibujos a la vez. Esa es la ventaja de tener padre e hijos.

Rojo · Adelanto de la práctica 09. Añade a `Mundo` un nodo `StaticBody2D` con su propio `CollisionShape2D` en medio de la pantalla, y comprueba que ahora el personaje ya no puede atravesarlo. No hace falta que le pongas dibujo: con la forma de colisión se ve en el editor. Si lo consigues, has hecho tu primer obstáculo.

---

10. Autoevaluación

- [ ] Sé explicar la diferencia entre un `Sprite2D` y un `CharacterBody2D` .
- [ ] Sé cuáles son los tres tipos de cuerpo y para qué sirve cada uno.
- [ ] Sé montar la estructura cuerpo, imagen y forma de colisión.
- [ ] Sé por qué la posición de un hijo se cuenta desde su padre.
- [ ] Sé qué hace `move_and_slide()` y por qué ya no necesito `delta` .
- [ ] Sé cuándo usar `_physics_process` en vez de `_process` .

---

11. Vocabulario

Español	Inglés	Qué es
Cuerpo	Body	Nodo que ocupa espacio y puede chocar
Forma de colisión	Collision shape	El contorno que de verdad choca
Cuerpo estático	StaticBody2D	El que no se mueve pero frena
Cuerpo de personaje	CharacterBody2D	El que mueves tú por código
Velocidad	Velocity	Propiedad con la velocidad deseada del cuerpo
Deslizar	Slide	Resbalar a lo largo de un obstáculo en vez de clavarse

12. Entrega

Comprime la carpeta como `apellido_practica06.zip` y súbela al Aula Virtual de Alexia.

Capturas necesarias en esta práctica

Código	Contenido
06.1	Editor con el contorno azul de colisión y el árbol de tres nodos
06.2	Aviso amarillo del cuerpo sin forma de colisión, con el mensaje abierto
06.3	Inspector con la propiedad Forma y la opción Nuevo RectangleShape2D